```
Output:
array(3) {
  ["name"]=>
  &string(3) "Bob"
  ["age"]=>
  &string(11) "Forty-seven"
  ["favouriteToy"]=>
  &string(19) "Benelli M4 Tactical"
}
```

We can see that `$arr` has been updated by changing the values of the imported variables; note the output of `var_dump()` shows that each element now has a reference operator next to the type (`&string(7) "Russell"`, etc.). The expression `EXTR_REFS|EXTR_PREFIX_ALL` is a *bitwise OR* operation, and you can find an explanation in Chapter 7, "Numbers": it's quite a common pattern seen in PHP functions.

Extreme caution is advised if using `extract()`, especially in global scope. One slip and critical data might be lost, or the global symbol table becomes polluted with unnecessary variables. It should also never be used on superglobal data (`$_POST`, `$_GET`, et al.) because the values in those special arrays can contain unfiltered user input, which should always be handled with care (and suspicion). Removal of this function from the language has been suggested on at least one occasion on the PHP *#internals* mailing list (https://externals.io/message/100637).

Fetching a Subset of Elements with `array_slice()`

To extract a sub-set of elements from an array, we can use the built-in `array_slice()`. This function will return sequential elements of an array starting at an offset parameter and ending once the required length parameter is satisfied.

```
$myArr1 = ["Alice", "Rita", "Sue", "Bob", "Russell"];

$myArr2 = array_slice($myArr1, 1, 3);

print_r($myArr2);
print_r($myArr1);
```